

MEMORIA TÉCNICA

Gestión de una Liga de E-sports

Autor: Hugo García Ruiz

Asignatura: Programación

Fecha: 15 Mayo 2026



1. DESCRIPCIÓN GENERAL DEL PROYECTO

1.1. Contexto y objetivos

El presente proyecto consiste en el desarrollo de una aplicación en Java para la gestión integral de una liga de e-sports. La aplicación se ejecuta por consola mediante un sistema de menús interactivos que permiten administrar todos los aspectos de la competición: registro de personas (jugadores y entrenadores), equipos, plantillas, calendario, partidos, incidencias, sanciones, apuestas, estadísticas y clasificación. Se aplican todos los contenidos del curso: programación estructurada, modular, orientada a objetos, arrays, matrices, excepciones, clases abstractas, interfaces y estructuras dinámicas.

1.2. Funcionalidades obligatorias

Se ha implementado la gestión completa de personas (alta, listar, buscar por ID, **modificar datos** y eliminar), la gestión de equipos, y la gestión de plantillas con array fijo de cinco titulares y lista dinámica de suplentes. Se controla que no haya dos titulares con el mismo rol dentro del mismo equipo.

1.3. Calendario, cola y pila

El calendario usa una matriz bidimensional que almacena los puntos por equipo y jornada. Los partidos pendientes se gestionan con una cola FIFO: se pueden encolar, consultar el siguiente, mostrar todos y **vaciar la cola** cuando proceda. El historial de acciones usa una pila LIFO con capacidad de deshacer altas de personas y equipos de forma real.

1.4. Partidos, convocatorias e incidencias

Para disputar un partido, cada equipo debe superar la validación de convocatoria (método `validarConvocatoria()` en `Equipo`): exactamente 5 titulares, ninguno sancionado, sin roles repetidos. Si falla se lanza `ConvocatoriaInvalidaException` y el partido permanece en la cola. Las incidencias permiten registrar sanciones, expulsiones y otros eventos, impidiendo que un jugador sancionado entre en convocatoria. Se controla también que no se registre dos veces un mismo partido y que los IDs de partido no se dupliquen gracias a un `HashSet` específico.

1.5. Estadísticas, clasificación y apuestas

El sistema muestra estadísticas de jugadores (partidas, MVPs, rendimiento, sanción, coste) y de equipos (victorias, derrotas, puntos, diferencia, coste de plantilla). La clasificación se calcula automáticamente ordenando por victorias, diferencia de puntos y nombre. Como funcionalidad extra se implementó un sistema de apuestas con cuotas dinámicas que se resuelven al disputar el partido.

1.6. Robustez en la entrada de datos

Todos los métodos de lectura por teclado (`pedirInt`, `pedirDouble`) están protegidos con bloques `try-catch` que capturan `InputMismatchException`, evitando que el programa patee si el usuario introduce texto donde se espera un número. El bucle repite la solicitud hasta recibir un valor válido.

2. EXPLICACIÓN DEL DISEÑO DE CLASES

2.1. Clase abstracta PersonaLiga

Superclase abstracta de Jugador y Entrenador. Contiene los atributos comunes (id, nombre, nickname, edad, salarioBase), métodos concretos (mostrarResumen(), getters/setters, toString()) y el método abstracto calcularCosteMensual(), que cada subclase implementa con su propia lógica.

2.2. Interfaz Entrenable

Declara entrenar() y calcularRendimiento(). La implementa la clase Jugador. El uso de interfaz en lugar de herencia múltiple proporciona mayor flexibilidad y permite que otras clases la implementen en el futuro.

2.3. Clase Jugador

Extiende PersonaLiga e implementa Entrenable. Atributos específicos: rol (enum TOP/JUNGLE/MID/ADC/SUPPORT), nivelMecanico, nivelEstrategico, partidasJugadas, mvpsTotales y sancionado. calcularCosteMensual() añade 100€ por MVP y 300€ si tiene 500+ partidas. calcularRendimiento() devuelve la media de niveles (máx. 10) con bonus de 5 si MVPs/partidas ≥ 0.5 .

2.4. Clase Entrenador

Extiende PersonaLiga. Atributos: experiencia, especialidad, victoriasTotales. calcularCosteMensual() = salarioBase + (victorias $\times$ 10) + (experiencia $\times$ 100). Ejemplifica el polimorfismo junto con Jugador al compartir la colección de PersonaLiga.

2.5. Clase Equipo

Atributos: nombre, ciudad, entrenador, presupuesto, victorias, derrotas, puntosFavor, puntosContra, array Jugador[5] titulares, ArrayList suplentes. Implementa Comparable con compareTo() completo (victorias desc $\rightarrow$ diferencia de puntos desc $\rightarrow$ nombre alfa), coherente con el Comparator de mostrarClasificacion().

2.5.1. validarConvocatoria()

Verifica que los 5 huecos estén ocupados, que ningún titular esté sancionado y que no haya dos titulares con el mismo rol. Si alguna condición falla lanza ConvocatoriaInvalidaException. Se invoca con peek() antes del poll() en Liga, por lo que el partido permanece en cola si falla.

2.5.2. Gestión de plantilla

añadirTitular() valida rol duplicado y sanción (TitularInvalidoException). añadirSuplente() valida que no sea ya titular ni esté en el banquillo (SuplenteInvalidoException). promoverSuplente() mueve un jugador del banco a una posición concreta del array con sus validaciones.

2.6. Clase Partido

Almacena id, jornada, equipoLocal, equipoVisitante, puntuaciones, mvp y estado disputado. registrarResultado() valida partido no disputado, equipos distintos y MVP perteneciente a algún equipo, luego actualiza estadísticas de equipos y jugadores.

2.7. Clase Incidencia

Atributos: id, descripcion, tipo (enum SANCION/EXPULSION/ERROR_TECNICO/PARTIDO_APLAZADO), equipo y jugador. aplicarSancion() marca al jugador como sancionado si el tipo es SANCION o EXPULSION, lanzando JugadorInvalidoException si el jugador es nulo.

2.8. Clase Apuesta (funcionalidad extra)

Permite apostar sobre un partido pendiente. La cuota se calcula como $1 + (\text{derrotas}+1)/(\text{victorias}+1)$. Se resuelve automáticamente al disputar el partido, actualizando el saldo del administrador si el equipo apostado resultó ganador.

2.9. Clase Liga

Clase contenedora principal. Agrupa: ArrayList de equipos, personas, incidencias y apuestas; Queue (LinkedList) para cola FIFO; Stack para historial LIFO; HashSet idsRegistrados y HashSet nombresEquipos para duplicados en O(1); **HashSet idsPartidos** para evitar IDs de partido duplicados; matriz int[][] para el calendario y saldoAdministrador.

2.9.1. deshacerUltimaAccion() con reversión real

Analiza el prefijo de la cadena en el tope de la pila. Si empieza por 'Jug. Registrado:' elimina esa persona de la lista y del HashSet de IDs. Si empieza por 'Eq. Registrado:' elimina ese equipo y su nombre del HashSet. Para el resto informa de que la operación no es reversible automáticamente.

2.9.2. vaciarCola()

Llama a colaPartidos.clear(), registra la acción en el historial con el número de partidos eliminados e informa al usuario. Cubre el requisito del enunciado de vaciar la cola cuando proceda.

2.10. Clase Main

Contiene el menú principal y todos los submenús. Los métodos pedirInt() y pedirDouble() usan bucles while(true) con try-catch sobre InputMismatchException para que el programa no patee ante entradas incorrectas. El método modificarPersona() permite editar nombre, nickname, edad, salario y los atributos específicos del tipo concreto (Jugador o Entrenador) mediante instanceof. inicializarDatos() precarga 6 entrenadores, 30 jugadores, 6 suplentes, 6 equipos con plantillas completas y 5 partidos encolados.

3. JUSTIFICACIÓN DEL USO DE ESTRUCTURAS DINÁMICAS

3.1. Array fijo — titulares

Jugador[5] para los titulares. Un equipo de e-sports tiene exactamente 5 roles fijos e inmutables (TOP, JUNGLE, MID, ADC, SUPPORT). El array comunica esta invariante, permite acceso directo por índice y es más eficiente que una lista para un tamaño conocido y fijo.

3.2. ArrayList — suplentes y colecciones principales

El número de suplentes puede variar (fichajes, despidos, ascensos). ArrayList ofrece add(), remove(), iterator() y mantiene el orden de inserción. Se usa también para equipos, personas, incidencias y apuestas en Liga por la misma razón: tamaño desconocido en tiempo de diseño.

3.3. Queue FIFO — cola de partidos

Implementada con LinkedList. El comportamiento FIFO garantiza que los partidos se disputen en el orden programado. offer() encola, poll() extrae el primero, peek() consulta sin extraer. Se puede vaciar con clear() cuando proceda.

3.4. Stack LIFO — historial de acciones

Comportamiento LIFO ideal para deshacer: la última acción realizada es la primera que el usuario querrá revertir. push() registra, pop() deshace (con reversión real para altas), peek() consulta la última acción sin eliminarla.

3.5. HashSet — control de duplicados en O(1)

Tres HashSet: idsRegistrados (IDs de personas), nombresEquipos (nombres de equipos) e idsPartidos (IDs de partidos). La comprobación de existencia en un HashSet es O(1) frente al O(n) de una búsqueda lineal en lista, lo que es crucial en operaciones frecuentes de registro.

3.6. Matriz bidimensional — calendario

int[equipos][jornadas]. Cada celda almacena los puntos obtenidos (3 victoria, 1 empate, 0 derrota). La matriz es la representación más natural para datos tabulares bidimensionales y permite acceso directo por índice a cualquier celda equipo-jornada.

3.7. Iterator — recorridos seguros

Se usa Iterator en lugar de for-each en los métodos que eliminan elementos durante la iteración (eliminarPersona, eliminarEquipo, deshacerUltimaAccion). Su método remove() evita ConcurrentModificationException.

4. EXPLICACIÓN DE LAS EXCEPCIONES

4.1. Filosofía general

Todas las excepciones personalizadas extienden `Exception` (comprobadas). La lógica de negocio (Liga, Equipo, Partido, Incidencia) las lanza; Main las captura con try-catch y muestra mensajes claros al usuario. Cada excepción recibe un mensaje descriptivo en el constructor que se pasa a `super()` y se recupera con `getMessage()`.

4.2. Las 12 excepciones personalizadas

PersonaDuplicadaException

Lanzada en `registrarPersona()` cuando el ID ya existe en `idsRegistrados`.

PersonaNoEncontradaException

Lanzada en `eliminarPersona()` cuando no hay persona con el ID indicado.

EquipoDuplicadoException

Lanzada en `registrarEquipo()` cuando el nombre ya existe en `nombresEquipos`.

EquipoNoEncontradoException

Lanzada en `eliminarEquipo()` cuando no existe equipo con ese nombre.

TitularInvalidoException

Lanzada en `añadirTitular()` por rol duplicado o jugador sancionado, y en `promoverSuplente()` por posición fuera de rango u ocupada.

SuplenteInvalidoException

Lanzada en `añadirSuplente()` si ya es titular o ya está en el banco, y en `eliminarSuplente()` o `promoverSuplente()` si el jugador no está entre los suplentes.

PartidoDisputadoException

Lanzada en `encolarPartido()` y en `registrarResultado()` cuando el partido ya está disputado.

PartidoInvalidoException

Lanzada cuando los dos equipos son el mismo objeto, o cuando el ID de partido ya existe en el `HashSet idsPartidos` (nueva validación).

MvpInvalidoException

Lanzada en `registrarResultado()` si el jugador MVP no pertenece a ninguno de los dos equipos.

JugadorInvalidoException

Lanzada en `aplicarSancion()` cuando la incidencia es SANCION/EXPULSION pero el jugador es nulo.

SaldoInsuficienteException

Lanzada en `realizarApuesta()` cuando la cantidad supera el saldo del administrador.

ConvocatoriaInvalidaException

Lanzada por `validarConvocatoria()` en Equipo cuando: algún hueco de titulares está vacío, algún titular está sancionado, o hay dos titulares con el mismo rol. El partido no se saca de la

cola hasta que ambos equipos superen esta validación.

4.3. Separación de capas

Las clases de negocio lanzan excepciones pero no las capturan. Main contiene todos los bloques try-catch. Esta separación mejora la mantenibilidad: si se cambia la interfaz (p.ej. de consola a gráfica) solo cambia la capa de presentación, no la lógica.

5. CONCLUSIÓN

5.1. OOP y modularidad

Encapsulación (atributos privados + getters/setters), herencia (PersonaLiga → Jugador/Entrenador), polimorfismo (calcularCosteMensual(), colecciones de PersonaLiga) y abstracción (clase abstracta e interfaz Entrenable). El código está organizado en métodos con única responsabilidad y sin lógica de negocio en main.

5.2. Estructuras de datos

Array fijo para titulares, ArrayList para colecciones variables, Queue FIFO para partidos, Stack LIFO para historial, tres HashSets para duplicados en O(1) y matriz bidimensional para el calendario. Cada estructura fue elegida por su comportamiento específico.

5.3. Excepciones

12 excepciones personalizadas que cubren todas las situaciones irregulares previsibles. Control robusto de entrada de datos con InputMismatchException en pedirInt() y pedirDouble(), y comprobaciones de nulidad antes de operar sobre objetos.

5.4. Mejoras incorporadas en la revisión

Durante la revisión del proyecto se realizaron seis mejoras: (1) compareTo() de Equipo completado y coherente con la clasificación; (2) validarConvocatoria() con ConvocatoriaInvalidaException integrada en el flujo de disputa; (3) deshacerUltimaAccion() con reversión real de altas de personas y equipos; (4) modificarPersona() implementada en el menú de personas cubriendo todos los atributos específicos por tipo; (5) vaciarCola() añadida como opción del menú de cola; (6) HashSet idsPartidos para evitar IDs de partido duplicados, con validación en encolarPartido() mediante PartidoInvalidoException.

5.5. Funcionalidad extra — apuestas

El sistema de apuestas va más allá de lo exigido. Cuotas calculadas dinámicamente según el historial del equipo, validación de saldo, resolución automática al disputar el partido y registro completo en el historial y en la pila de acciones.

5.6. Valoración final

La aplicación es completamente funcional, robusta frente a entradas incorrectas, coherente en su diseño y demuestra el dominio de todos los contenidos del curso. Las seis mejoras de la revisión completan los requisitos obligatorios del enunciado y refuerzan la solidez del sistema tanto en corrección funcional como en experiencia de usuario.